

Recoverability-Gated Deliberation for Release-Aware Slow Reasoning in Closed-Loop Driving

LiYi Xie

Abstract—Large language models and vision–language models have demonstrated strong capabilities in semantic scene interpretation and high-level decision making for autonomous driving. However, their inference latency introduces a fundamental release-time mismatch: a reasoning request is issued from one traffic state, whereas its response becomes available only after the scene has evolved. Existing dual-process and asynchronous driving systems focus on when to invoke the slow reasoner or how to maintain fast control during inference, yet they lack an explicit mechanism to evaluate whether a delayed proposal remains valid, distinct from the current fast action, and beneficial in the evolved traffic state. Without such a mechanism, stale or redundant proposals may silently enter the control loop, while genuinely corrective actions may be discarded.

This paper presents Recoverability-Gated Deliberation (RGD), a two-stage framework that governs both slow-reasoner admission and release validation in closed-loop driving. At query time, RGD issues a request only when delay feasibility, action support, recovery-cost advantage, and scene demand are jointly satisfied. At release, the returned proposal is remapped in the observed state and authorized only after passing feasibility, action-distinctness, and recovery-cost checks against the current fast command. We further introduce Recoverable Value of Deliberation (R-VoD), an offline matched-rollout diagnostic that quantifies the corrective opportunity retained at release. Closed-loop experiments on highway-env and MetaDrive demonstrate that RGD substantially reduces slow-path invocations while preserving task outcomes under the evaluated paired protocol, and that the release gate retains the corrective intervention in the evaluated proposal stream while filtering harmful or negligible alternatives.

Index Terms—Autonomous driving, closed-loop decision making, inference latency, large language models, recoverability, selective computation.

I. INTRODUCTION

LARGE language models (LLMs) and vision–language models (VLMs) have advanced semantic scene interpretation, driving-knowledge retrieval, and high-level decision making in autonomous driving [1]–[8]. By providing long-horizon intent and contextual reasoning beyond reactive policies, these models offer a promising deliberative pathway. In closed-loop deployment, however, inference latency creates a release-time mismatch: a reasoning request is issued from one traffic state, whereas its proposal arrives in a later, potentially different state. Although a high-frequency controller can maintain vehicle control during asynchronous inference, the system must still decide whether the delayed proposal should influence the current command.

For example, consider a closing lead vehicle with an initially open adjacent lane. The system requests a lane-change recommendation, but by the time the response arrives, the target gap may have narrowed, the maneuver may no longer be admissible, or the fast controller may already execute the same

action. The release decision must therefore test the proposal against the current state and the concurrent fast action.

Maintaining a fast control stream alone does not resolve this problem. A returned action may pass an isolated feasibility check yet coincide with the fast action or offer no cost advantage. Properly evaluating a delayed proposal requires a common action schema, a shared mapper, and a state-dependent evaluator that compares both branches at the release state.

Existing work addresses parts of this process. Invocation policies use uncertainty, scene complexity, or policy discrepancy to decide when to engage a slow branch [9]–[11], and VLM-based attention allocation prioritizes surrounding vehicles under behavioral uncertainty [12]. Asynchronous architectures preserve fast control during model inference [8], [13], while DualDrive aligns semantic intent with a real-time policy [14]. Runtime-assurance methods guard whether a candidate action may enter the actuation path [15]. However, these approaches address invocation timing, control continuity, or candidate safety in isolation. The critical gap remains: to our knowledge, existing methods have not jointly provided a release-time mechanism that remaps a delayed proposal in the evolved state, compares it with the concurrent fast action through a shared interface, and decides whether the proposal still offers a corrective advantage.

This paper addresses both questions within a single framework. Two constraints guide the design: the fast controller must remain active throughout inference, and rejected proposals must not disrupt the control loop. RGD treats the slow reasoner as a proposal generator rather than a replacement for the fast controller. The fast action remains the default; a slow proposal reaches actuation only through a shared mapper and safety interface, which also permits reasoner substitution without altering the release logic.

A release state is recoverable when at least one feasible alternative remains distinct from the fast action and exceeds a matched-rollout utility margin. Actionability is evaluated at release: the returned command must pass feasibility checks, differ from the mapped fast command, and satisfy the recovery-cost margin. Recoverable Value of Deliberation (R-VoD) quantifies recoverability offline through matched rollouts under a common continuation policy, measuring the corrective opportunity that persists at release.

At query time, the online gate requires delay feasibility, action support, cost advantage, and scene demand simultaneously; high urgency alone cannot compensate for an expired maneuver window. When a response arrives, release validation evaluates the proposal in the current state and accepts it only if it passes feasibility, distinctness, and recovery-cost checks.

We evaluate RGD through paired allocation experiments

across six policies, controlled-delay traces, ablation studies, matched release-state interventions, reasoner substitution, and additional scenario-family evaluations.

The main contributions are as follows:

- 1) We formalize delayed deliberation through two release-time concepts, recoverability and actionability, and define R-VoD as an offline matched-rollout diagnostic of the corrective opportunity at release.
- 2) We develop RGD, a two-stage framework whose conjunctive query admission and release gate evaluate both branches through a shared action interface in the observed release state.
- 3) We conduct closed-loop experiments on highway-env and MetaDrive demonstrating that RGD substantially reduces slow-path invocations while preserving task outcomes, and that the release gate retains the corrective intervention in the evaluated proposal stream while filtering harmful or negligible alternatives.

II. RELATED WORK

A. Language-Conditioned and Generative Driving

Language-conditioned driving methods differ in the proximity of their outputs to vehicle actuation. At the advisory level, DiLu retrieves relevant experience before producing an interpretable decision [1], and Driving with LLMs represents traffic through object-level vectors and returns actions with explanations [16]. VLM-based systems combine visual observations with language reasoning for navigation and motion planning [6], [17], while knowledge retrieval methods supply domain-specific context before decision making [7]. These systems use language-conditioned interfaces to encode scene relations and retrieve driving knowledge.

A second group places model outputs closer to the control boundary. Policy transfer methods use language models to support vehicle decision making [18], GPT-Driver casts planning as language modeling [5], and C-TRAIL integrates commonsense reasoning with trajectory planning [19]. Generative driving methods learn predictive scene representations or future trajectories for downstream planning [20]–[23]. As outputs approach actuation, a delayed proposal may have fewer downstream opportunities for correction and should be evaluated in the state in which it will execute.

These studies focus primarily on model capability or driving representation. By contrast, RGD operates at the interface between a slow proposal generator and the fast controller, evaluating whether a delayed proposal should replace the current fast action in the state where it will actually execute.

B. Dual Process Reasoning and Selective Computation

Dual-process driving systems divide responsibility between a responsive policy and a deliberative model. LeapAD uses reflection and accumulated experience, LeapVAD adds cognitive perception, and FASIONAD coordinates both branches through adaptive feedback [9], [24], [25]. DualDrive aligns semantic intent from a VLM with a real-time policy without relying on buffering or asynchronous updates [14], and NetRoller

connects general and specialized models through asynchronous execution with explicit handling of differing model frequencies [8]. Both designs keep a fast branch active while the deliberative component contributes at a system-defined rate.

Selective computation methods determine when the slow branch should run. AdaDrive adapts deliberation depth to scene demand [10], and AdaThinkDrive selects between fast direct answers and slow chain-of-thought reasoning modes according to scene complexity [26]. G-VLM uses behavior discrepancy between lightweight and multimodal policies as a self-supervised training signal for an online invocation gate [11]. Behavioral Attention allocates VLM attention among surrounding vehicles under behavioral uncertainty [12], addressing perceptual resource management rather than slow-path invocation timing. These signals are evaluated before inference begins, when neither the returned action nor the release state is known. By contrast, RGD supplements query-time selection with an explicit release-time evaluation against the concurrent fast action, ensuring that a successfully issued request does not automatically receive release authority.

Metareasoning balances expected decision improvement against computational cost. Time-bounded and anytime methods schedule reasoning under a finite budget [27]–[30], and concurrent planning permits an executor to continue acting while a planner computes [31]. In driving, an estimate made at query time can differ from the opportunity available at release because traffic evolves while the fast controller continues to act. R-VoD measures the remaining corrective opportunity through matched rollouts.

C. Runtime Safety, Delay, and Viability

Timing mechanisms preserve control during inference. AsyncDriver separates language-model planning from real-time control [13], NetRoller keeps a specialized driving model active [8], event-triggered control adjusts the timing of stabilizing control updates [32], and edge-computing frameworks schedule offloaded computation across heterogeneous resources [33]. Delay-aware control further models state evolution during input lag [34]. At the assurance layer, Simplex-based systems revert to a trusted controller when a learned action fails an acceptance test [15], and formal safety and viability methods characterize admissible motion under dynamic constraints: RSS defines responsibility-sensitive safety distances [35], and viability theory provides invariance certificates for continuous dynamics [36].

These approaches address timing, invocation, or candidate safety individually but do not compare the delayed proposal with the concurrent fast action as a distinct release-time decision. RGD complements runtime safety by adding an explicit release-time evaluation: beyond checking whether a mapped action can enter actuation, the release gate tests whether the proposal is distinct from, and offers a recovery-cost advantage over, the fast action in the evolved state. The final safety projection remains active after selection.

III. RECOVERABILITY-GATED DELIBERATION

RGD decomposes slow-reasoner invocation into two sequential authority decisions: query admission and release

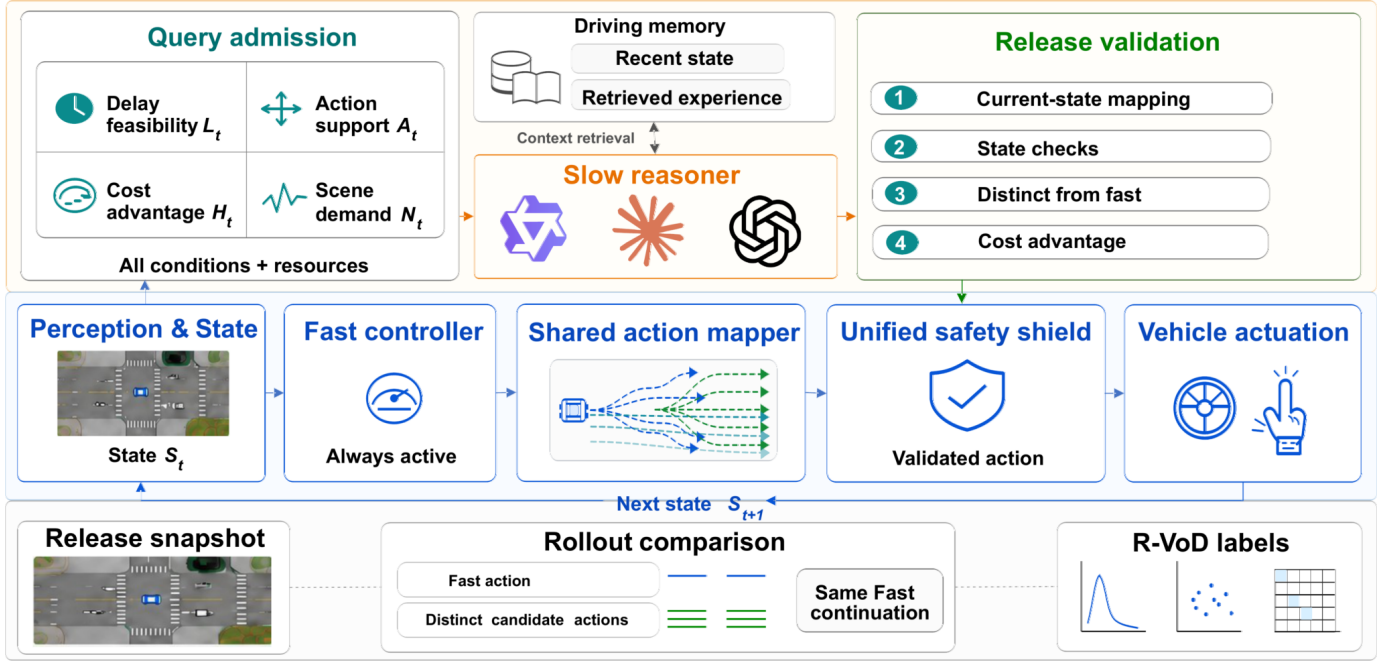


Fig. 1. RGD system architecture. Solid arrows: online control paths; dashed arrow: offline R-VoD diagnostic branch. The memory module belongs to the slow-reasoner input context, not to the online gate.

validation. Admission determines whether the current scene warrants deliberation and whether a feasible alternative can be supported within the expected response interval. Release validation evaluates the returned proposal against the observed state at the time of delivery. The fast controller remains the default throughout inference and is retained whenever either stage withholds slow-path authority.

A release state s_r is *recoverable* if it admits at least one feasible effective alternative that is distinct from the fast effective action and exceeds the R-VoD margin. R-VoD quantifies this corrective opportunity offline via matched rollouts. A returned proposal is *actionable* at release only if its mapped command passes current-state feasibility, differs from the mapped fast command, and satisfies the proposal-specific recovery-cost margin. The final safety projection may subsequently collapse an authorized slow command to the same executable action as the fast command; R-VoD and release validation thus serve complementary, not interchangeable, roles.

Fig. 1 illustrates the two online gates and the offline R-VoD diagnostic branch. The fast controller remains active at every control cycle; a missing response, mapping failure, or rejected release leaves the fast command unchanged. Both branch outputs are projected into the same mapped command space prior to the shared feasibility and cost interfaces, ensuring that semantically equivalent expressions resolve to the same maneuver irrespective of reasoner phrasing.

A. Query and Release Process

At frame t , the ego vehicle observes state s_t . The fast policy produces an action that the shared mapper converts to a_t^f . Both branches share this mapper and the same final safety projection $\Phi(s, a)$, which maps a command to an executable action in state s . Branch outputs are parsed into a

maneuver and a target speed; two expressions are considered equivalent if they induce the same mapped behavior. The high-level action domain comprises *lane-left*, *lane-hold*, *lane-right*, *accelerate*, and *decelerate*, all resolved through the same simulator interface. The response parser $\psi(y)$ normalizes a structured action identifier or canonical token; $\mathcal{M}(s, y) = \psi(y)$ if and only if $\psi(y) \in \mathcal{A}(s)$, and is undefined otherwise. This constitutes current-state semantic mapping rather than trajectory replanning.

The returned proposal is remapped in the observed release state s_r . A lane-change or speed command that was feasible at query time may no longer be valid; any mapping or feasibility failure causes a_r^f to be retained.

The delay estimate is used solely at admission. The actual release frame r is the first control frame at which the response becomes available, and both the returned command and the fast command are evaluated in the observed state s_r . Responses that do not arrive before episode termination are discarded.

B. Offline Measure of Corrective Opportunity

Offline matched rollouts provide a state-level diagnostic of the corrective opportunity available at release, independent of any specific slow-reasoner response. At each logged release snapshot s , let $a_{\text{eff}}^f = \Phi(s, a^f)$ denote the effective fast first action under the shared mapper and final safety projection. Let $\mathcal{D}(s)$ be the set of feasible effective first actions that remain distinct from a_{eff}^f after the same projection stages, and let $\mathcal{C}(s) = \{a_{\text{eff}}^f\} \cup \mathcal{D}(s)$. The matched-rollout utility $J_H(s, a)$ for horizon $H \in \mathbb{N}_+$ and discount $\gamma \in [0, 1]$ is defined as

$$J_H(s, a) = \frac{\sum_{k=0}^{H-1} \gamma^k r_k}{\sum_{k=0}^{H-1} \gamma^k} - \mathbb{I}[E_C], \quad (1)$$

where $s_0 = s$, $a_0 = a$, the fast policy supplies a_k for $k \geq 1$, r_k is the per-step environment reward, and E_C is the event that a collision occurs within the rollout horizon; a collision terminates the rollout and incurs a unit penalty. We set $H = 20$ and $\gamma = 0.99$; candidates sharing the same projected command and target speed are merged prior to evaluation. The Recoverable Value of Deliberation (R-VoD) is then defined as

$$\text{R-VoD}(s) = \max_{a \in \mathcal{C}(s)} [J_H(s, a) - J_H(s, a_{\text{eff}}^f)]. \quad (2)$$

Since $a_{\text{eff}}^f \in \mathcal{C}(s)$, R-VoD is nonnegative by construction and equals zero when no alternative improves upon the fast continuation. A state is labeled *corrective* when $\text{R-VoD}(s) \geq \epsilon_R$, where $\epsilon_R = 0.02$ is a fixed utility-scale margin.

By varying only the first effective action while holding the remainder of the trajectory under the common fast policy, R-VoD isolates the corrective opportunity at release from any downstream dependence on the slow-reasoner response.

C. Query Admission

Let $\mathcal{A}(s_t)$ denote the exact runtime action domain and a_t^f the mapped fast command. The runtime recovery-cost evaluator is

$$c_t(a) = \text{clip}_{[0,1]} \left(h_t + \left[d_t(a) - \min_{b \in \mathcal{A}(s_t)} d_t(b) \right]_+ \right), \quad (3)$$

where $[x]_+ = \max(x, 0)$; $d_t(a) \in [0, 1]$ is the fixed-weight penalty combining safety, comfort, and efficiency costs; and $h_t \in [0, 1]$ is the scene-level hazard aggregating TTC urgency, headway deficit, obstacle proximity, and interaction pressure. Lower values of c_t are preferred. Malformed cost inputs or inconsistent action domains suppress admission. The domain integrity guard F_t requires consistent action-domain agreement and a nonempty feasible-alternative set $\mathcal{D}_t = \{a \in \mathcal{A}(s_t) \setminus \{a_t^f\} : c_t(a) \leq \kappa_c\}$, where $\kappa_c > 0$ is the recovery-cost feasibility threshold; $F_t = 0$ and no slow request is issued whenever $\mathcal{D}_t = \emptyset$.

Delay feasibility is characterized by control frequency f , predicted latency $\hat{\tau}_t$, critical maneuver window T_t^{crit} (estimated from the current TTC and admissible trajectory corridor), and safety reserve ρ . The residual-window score is

$$\ell_t = \text{clip}_{[0,1]} \left(\frac{T_t^{\text{crit}} - \lceil f\hat{\tau}_t \rceil / f - \rho}{T_t^{\text{crit}}} \right). \quad (4)$$

Unavailable timing evidence sets $L_t = 0$. Let $\mathcal{F}$ denote the fixed set of non-Fast maneuver families, $\mathcal{F}_t \subseteq \mathcal{F}$ those represented in $\mathcal{D}_t$, u_m the minimum recovery cost among members of family m in $\mathcal{D}_t$, and $u_* = \min_{m \in \mathcal{F}_t} u_m$. Normalizing by $|\mathcal{F}|$ scores family coverage as an absolute fraction of all families, so that absent families penalize the score; $\mathcal{F}_t = \emptyset$ sets $A_t = 0$. The four admission tests are

$$L_t = \mathbb{I}[\ell_t \geq \lambda_L], \quad (5a)$$

$$A_t = \mathbb{I} \left[|\mathcal{F}|^{-1} \sum_{m \in \mathcal{F}_t} e^{-(u_m - u_*)/T_A} \geq \lambda_A \right], \quad (5b)$$

$$H_t = \mathbb{I} \left[\kappa_c^{-1} [c_t(a_t^f) - \min_{a \in \mathcal{D}_t} c_t(a)]_+ \geq \lambda_H \right], \quad (5c)$$

$$N_t = \mathbb{I}[\max(h_t, n_t^{\text{pre}}) \geq \lambda_N], \quad (5d)$$

where $n_t^{\text{pre}} \in [0, 1]$ is an independent scene-hazard pre-screen score derived from obstacle density and conflict geometry, and $T_A > 0$ is the soft-aggregation temperature for family support. All thresholds $(\lambda_L, \lambda_A, \lambda_H, \lambda_N)$ are selected on the development cohort and frozen prior to the reported evaluation; all four conditions must hold simultaneously.

Let $v_t \in \{0, 1\}$ encode resource availability, including request budget, cooldown, and executor status, subject to the requirement that no slow request is already pending. The admission rule is

$$q_t = \mathbb{I}[v_t \wedge F_t \wedge L_t \wedge A_t \wedge H_t \wedge N_t], \quad (6)$$

where $\mathbb{I}[\cdot]$ is the indicator function.

D. Release Validation

At release frame r , let y_r denote the returned response and $\mathcal{M}$ the shared current-state mapper. The fast policy independently produces a_r^f from the same state s_r . If $\mathcal{M}(s_r, y_r)$ is undefined, the response is rejected and a_r^f is retained. Otherwise, let $\tilde{a}_r^{\text{sl}} = \mathcal{M}(s_r, y_r)$ denote the mapped slow command. The state-level predicates F_r , A_r , H_r and the cost function c_r are recomputed in s_r ; delay feasibility and scene demand are admission-only conditions that do not re-enter release. The feasible slow-command set is defined as $\mathcal{V}_r = \{a \in \mathcal{A}(s_r) : c_r(a) \leq \kappa_c\}$ if $F_r \wedge A_r \wedge H_r$ holds, and $\mathcal{V}_r = \emptyset$ otherwise. The release indicator is

$$g_r = \mathbb{I}[\tilde{a}_r^{\text{sl}} \in \mathcal{V}_r \wedge \tilde{a}_r^{\text{sl}} \neq a_r^f \wedge c_r(\tilde{a}_r^{\text{sl}}) + \delta \leq c_r(a_r^f)], \quad (7)$$

where $\delta \geq 0$ is the release margin (expressed on the same scale as c_r), fixed at $\delta = 0.02$.

The selected and executed commands are accordingly

$$a_r^{\text{sel}} = \begin{cases} \tilde{a}_r^{\text{sl}}, & g_r = 1, \\ a_r^f, & g_r = 0, \end{cases} \quad a_r^{\text{exec}} = \Phi(s_r, a_r^{\text{sel}}). \quad (8)$$

Equation (8) makes the fast-path fallback explicit: whenever $g_r = 0$, $a_r^{\text{exec}} = \Phi(s_r, a_r^f)$. Distinctness is evaluated between mapped commands prior to the projection Φ ; consequently, an authorized slow command may still map to the same executable action as the fast command after projection, and this post-projection outcome is reported separately. Query admission confers no persistent authority on a returned proposal, and no R-VoD label enters the online release decision. All checks operate on the observed state s_r and authorize only proposals that provide a strict recovery-cost advantage over the concurrent fast action.

IV. EXPERIMENTS AND DISCUSSION

A. Experimental Setup

RGD is evaluated on highway-env [37] and MetaDrive [38] closed-loop simulators with Qwen3-8B [39] as the default slow reasoner. The main highway-env experiments run at 10 Hz for 30 s per episode. Within each comparison, all methods share the seed cohort, action schema, fast controller, slow-reasoner interface, request budget, and evaluation rule. Gate development, main evaluation, and mechanism analysis use disjoint cohorts. The paired allocation study uses 30 seeds and

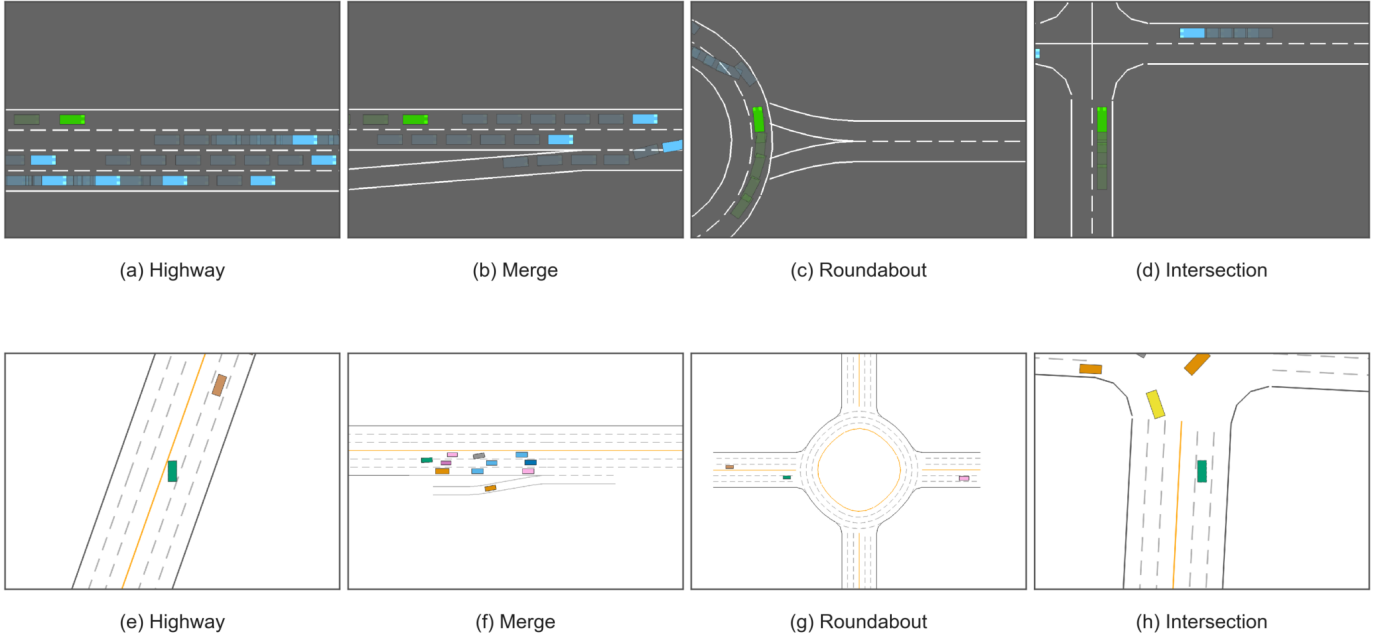


Fig. 2. Representative highway-env and MetaDrive scenes.

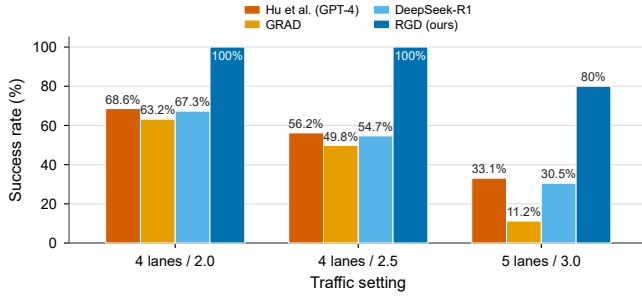


Fig. 3. Success rates across the three highway-env traffic settings of Hu et al. [6].

six policies, yielding 180 policy–seed units. Each component study draws from a separate 20-seed cohort; the six-condition ablation study contains 120 paired condition–seed units.

The simulator seed serves as the experimental and bootstrap unit. We report 95% percentile bootstrap intervals from 20,000 resamples. Binary paired endpoints use exact McNemar tests, continuous endpoints use paired sign-flip tests, and the five prespecified RGD–baseline contrasts are Holm-adjusted within each endpoint. R-VoD is computed after release through matched first-action rollouts. Gate criteria, corrective margins, and the service-latency stratum are selected on the development or calibration cohorts and remain frozen throughout the reported evaluation.

The analyses span six properties: published-settings context, admission selectivity under delay sweep, release-gate validation, gate criterion ablations, reasoner substitution, and cross-scenario transfer. The gate thresholds are selected on a held-out development cohort (seeds 2000–2039) across three delay strata (0.7, 1.7, 2.7 s), maximizing the corrective-state margin while minimizing requests per corrective state.

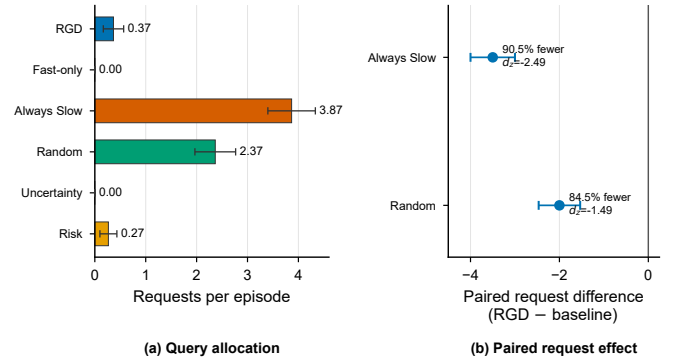


Fig. 4. Slow-path request counts and paired RGD-minus-baseline effects under the 30-seed common protocol.

The paired allocation cohort provides the primary resource and task-endpoint comparison. Separate controlled-delay cohorts support event-level analysis of timing, validation, and projection. To isolate release validation, a matched audit freezes the 11 proposals naturally issued by RGD on the 30 paired seeds; the release-validated and no-validation arms receive the same proposals and release timing. Every post-projection distinct action is compared with Fast from its matched online release snapshot using the matched-rollout utility (1).

Fig. 2 shows representative highway-env and MetaDrive scenes. The upper row covers highway, merge, roundabout, and intersection geometries; the lower row shows the corresponding MetaDrive reconstructions.

B. Performance Context Under Published Settings

Under the environment settings of PADriver [40], Table I places RGD within the operating ranges of the reported methods. RGD completes 29 of 30 runs at 611 m and 73.60 km/h, operating at 37.6 ms per frame without dedicated training on the evaluation environment. Following the source

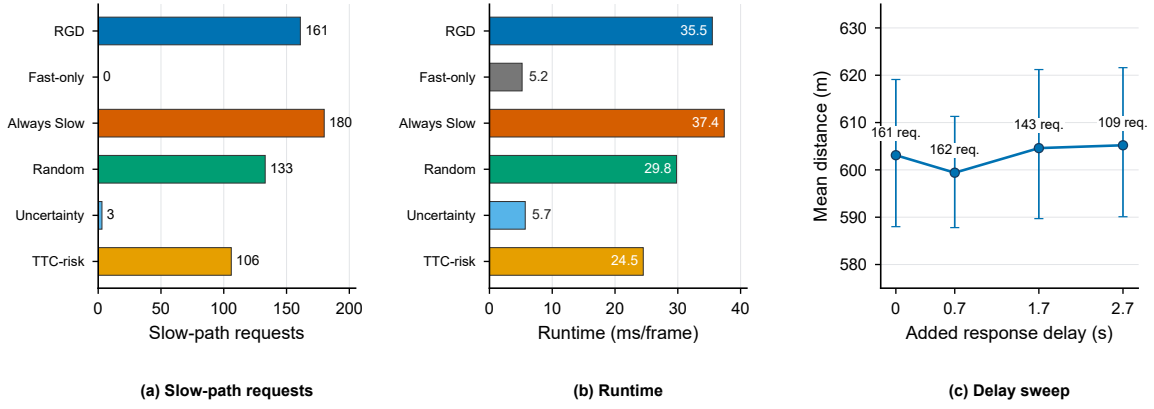


Fig. 5. Request counts, per-frame runtime, and mean travel distance under zero added delay and across the delay sweep. (a) Total slow-path requests in the zero-delay cohort, (b) mean per-frame runtime (ms), and (c) mean distance (m) with 95% bootstrap intervals; panel (c) labels indicate request counts.

TABLE I
PERFORMANCE CONTEXT UNDER PADRIVER-REPORTED
ENVIRONMENT SETTINGS.

Method	Dist. (m)	Speed (km/h)	Saf.	Kep.	Succ. (/30)	Time (s/fr.)
Driving with LLMs	411	49.42	0.49	0.56	20	48.0
GPT-Driver	428	51.40	0.45	0.52	23	36.2
DiLu	386	46.29	0.60	0.72	28	24.1
PADriver-Normal	603	72.47	0.91	0.92	25	1.4
RGD (ours)	611	73.60	0.75	0.86	29	0.038

definitions, Saf. denotes safe-distance-keeping rate and Kep. denotes lane-keep rate; both are reported in $[0, 1]$, with larger values indicating a higher frequency of the corresponding event.

Hu et al. [6] define success as 30 consecutive collision-free frames. We adopt their three traffic settings and ten seeds per setting with no added delay. Fig. 3 reports success rates alongside the published values. Table II provides RGD's operating metrics under the settings stated in the source paper.

Under the settings stated by Hu et al., RGD attains 100% success at 4L/2.0 and 4L/2.5. At 5L/3.0, it attains 80% success and schedules 0.8 slow-path requests per episode. Table II reports safety-qualified speed, assigns zero to unsuccessful episodes, and averages requests over the ten episodes in each setting. The per-frame runtime remains below 66 ms in all three settings.

C. Delay-Controlled Allocation and Component Evidence

1) *Paired Common-Protocol Allocation*: The paired allocation study compares RGD against five scheduling baselines under a common 30-seed protocol: Fast-only (never invokes the slow reasoner), Always Slow (issues a request at every eligible frame), Random (requests with uniform probability at each eligible frame), Uncertainty (requests when the fast policy entropy exceeds a threshold), and Risk (requests when the current TTC-urgency score h_t exceeds a threshold). All methods share the same Qwen3-8B service, fast controller, action mapper, release interface, and episode realization; only the slow-path allocation policy differs. Fig. 4(a) reports requests per episode, and panel (b) reports the paired RGD-minus-baseline effects for the two request-intensive allocators.

RGD averages 0.37 requests per episode, reducing slow-path use by 90.5% relative to Always Slow and by 84.5% relative to Random. The paired differences are -3.50 requests

TABLE II
RGD OPERATING METRICS UNDER THE SETTINGS OF
HU ET AL.

Setting	Dist. (m)	Speed (m/s)	Req/ep.	Time (ms/frame)
4L / 2.0	69.29	23.84	1.0	58.9
4L / 2.5	66.19	22.76	1.0	65.2
5L / 3.0	52.38	17.38	0.8	50.1

[95% CI $-4.00, -3.00$] and -2.00 requests [95% CI $-2.47, -1.53$], with standardized effects $d_z = -2.49$ and -1.49 , respectively. Both contrasts remain significant after Holm correction ($p_H = 0.00025$). Fast-only and Uncertainty provide zero-request references without engaging the slow reasoner.

2) *Zero Added Delay Reference*: The zero-delay cohort holds the fast controller, slow reasoner, request budget, and feasibility interface fixed across all six allocation policies; only the request and release rules differ. The delay sweep then uses the same seeds and opportunity schedule to assess delay-feasibility effects on slow-path allocation.

Without added delay, Fig. 5(a)–(b) shows that RGD schedules fewer requests and achieves lower per-frame runtime than Always Slow. Across the delay sweep in Fig. 5(c), request counts remain similar at 0 and 0.7 s and decrease at 1.7 and 2.7 s while distance intervals overlap, consistent with the delay-feasibility condition inducing greater selectivity without per-delay retuning.

3) *Allocation With a 1.7-s Added Delay*: The 1.7-s experiment examines how RGD processes a slow response after the traffic state has evolved. The fast controller, slow reasoner, request budget, seed cohort, and feasibility interface remain fixed. Fig. 6 reports the response-validation stages and the resulting release decisions.

Each returned proposal arrives in a state later than the state that initiated the request. RGD applies a returned proposal only after all release checks pass.

4) *Response Validation and Component Effects*: Response traces follow each request from admission through response validation, current-state mapping, and actuation. At the same 1.7-s added delay, the one-at-a-time ablation variants draw from a separate 20-seed cohort; each removes one admission criterion while retaining the release procedure.

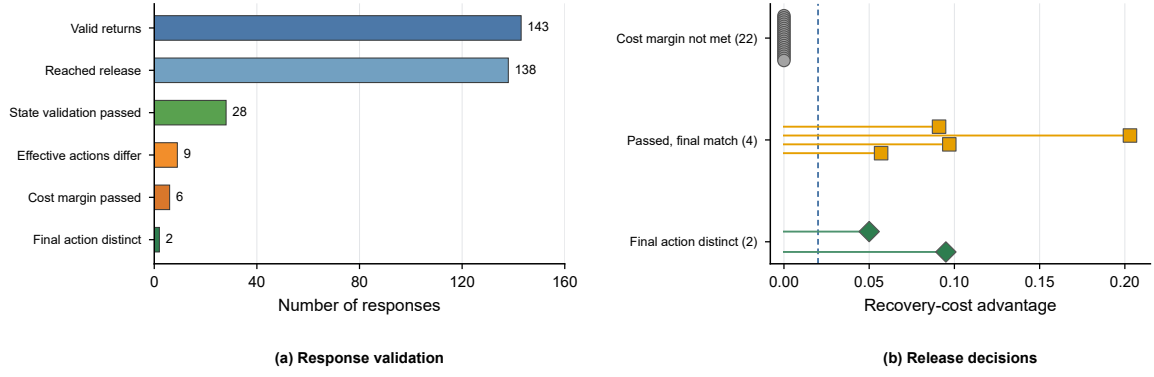


Fig. 6. Response validation pipeline under 1.7-s added delay. (a) Stage-wise proposal retention and release outcomes from valid response return through post-projection actuation. (b) Release-decision outcomes stratified by recovery-cost advantage.

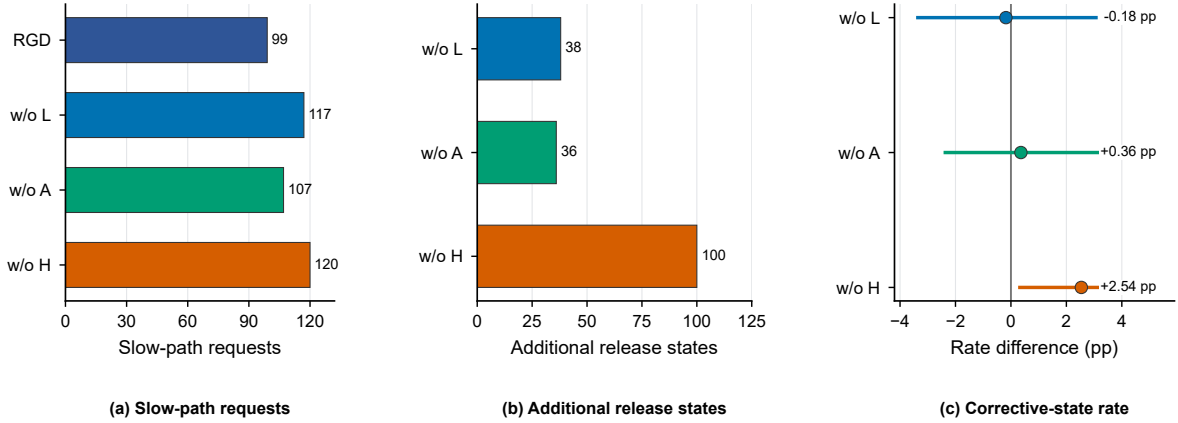


Fig. 7. One-at-a-time admission-criterion ablation. (a) Total requests over the 20-seed cohort, (b) additional release states, and (c) corrective-state-rate differences (ablation minus Full RGD) with 95% bootstrap intervals. Removing H produces the largest increase in corrective-state rate; removing L or A primarily shifts request volume and release-stage counts.

Under 1.7-s delay, 143 RGD requests return valid responses and 138 reach release before episode termination. Current-state checks admit 28 responses to release comparison; nine mapped slow commands differ from the concurrent fast command, and six satisfy the recovery-cost margin. After the shared final projection Φ , four of these authorized commands match the projected fast command and two remain distinct. Fig. 6(a) reports the validation stages from valid response return; panel (b) summarizes all 28 state-valid outcomes. A delayed proposal affects the final command only after passing all release checks and remaining distinct after projection; otherwise the fast action stays in control.

Fig. 7(a)–(b) shows that removing L , A , or H changes both the number of requests and the additional release states relative to Full RGD. Panel (c) reports the ablation-minus-Full-RGD difference in R-VoD-labeled corrective-state rate with 95% seed-bootstrap confidence intervals. The H contrast has a confidence interval entirely above zero, directly linking the online cost-advantage condition to the corrective-state rate of the selected cohort. The request and additional-release-state changes in panels (a)–(b) further show that L and A shape which states are sent to the slow path. Together, these analyses confirm the complementary roles of delay feasibility, action support, and recovery-cost advantage in the admission rule.

The corrective-state rate in Fig. 7(c) is labeled offline by

R-VoD, whereas H_t is computed online. The H contrast thus links an online admission signal to a separately constructed release-state diagnostic.

5) Full-Gate and Projection Analysis: Whereas the preceding ablation tests admission-criterion effects under natural request flow, this analysis replays a shared proposal bank to isolate gate and projection factors independently of scheduling variation. Six conditions are evaluated: Full RGD, removal of each of L , A , H , and N , and joint removal of H and N . All conditions replay the same gate-independent proposal source over 20 held-out seeds at a fixed 1.7-s delay. The action mapper, release rule, safety projection, budget, and cooldown remain unchanged, so each comparison removes only the named admission predicate.

The shared bank is frozen from a Qwen3-8B Always-Slow source that does not read the evaluated gate. Each record binds the source frame, raw action, response outcome, measured latency, and response hash; all arms replay the same bank and recompute the Fast action at every frame. Gate decisions are completed before proposal fields are exposed to an arm.

Fig. 8(a) traces the 70 synchronized candidates through the serial admission stages. Full RGD retains 49 candidates after L , 40 after A , eight after H , and none after N . Removing L retains 70 candidates at the first stage and 61 after A ; removing A retains 49 after the second stage. These trajectories expose

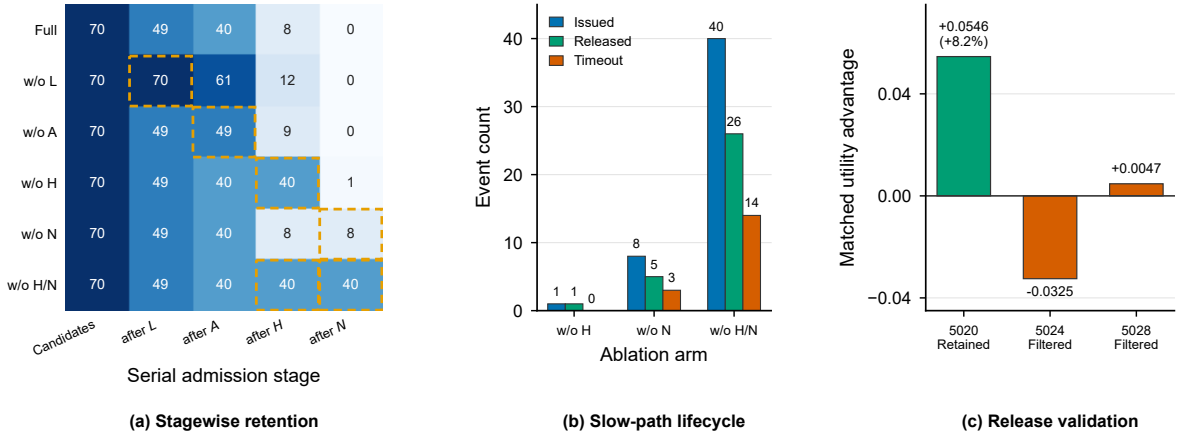


Fig. 8. Six-condition gate ablation and matched-proposal release audit. (a) Serial candidate retention across admission stages; dashed cells mark omitted predicates. (b) Slow-path processing volume over 20 paired seeds. (c) Matched-rollout utility advantage for the three post-projection distinct interventions.

TABLE III

RESULTS WITH DIFFERENT SLOW REASONERS.

Reasoner	Valid Resp.	Distance (m)	Completion (/30)
Qwen3-8B	138	599.16	29
Qwen3.5-4B	138	599.16	29
Qwen2.5-7B	122	599.12	29
GPT-5.6-sol	112	603.39	29
GPT-5.6-terra	132	599.16	29
GPT-5.6-luna	133	599.16	29
Fable-5	137	599.16	29

TABLE IV

RESULTS ACROSS LANE AND DENSITY SETTINGS.

Lanes/Density	Distance (m)	Successful Runs (/30)
4L/2.0	599	29
4L/3.0	478	23
5L/2.0	629	30
5L/3.0	429	20
6L/2.0	611	28
6L/3.0	453	20

the distinct upstream screening roles of L and A . Removing H , N , and H/N leaves 1, 8, and 40 candidates after the complete gate, respectively. Panel (b) follows these three conditions through the slow-path processing stages: 1, 8, and 40 requests are issued, of which 1, 5, and 26 reach delayed release. This analysis separates candidate reduction by L/A from the final workload control provided by H/N .

The command-level trace separates release exposure from executable effect. Removing N and H/N yields 4 and 16 mapped proposals distinct from the fast command. The shared final projection maps all of them to the fast-equivalent executable action, and task endpoints remain invariant across all 120 paired condition–seed units. This ablation identifies the dominant $H \times N$ interaction in slow-path workload and confirms that the fast-path fallback is preserved at the execution boundary.

The matched-proposal release analysis replays the 11 natural RGD proposals through the current release gate. Both release-validated and no-validation conditions obtain nine releases. Without validation, three returned commands remain distinct after the common projection; release validation retains one. Fig. 8(c) reports their matched utility. The retained action improves the utility (1) from 0.667 to 0.721, an 8.2% relative gain, and increases 20-step progress by 23.70 m without a collision in either branch. The two filtered actions have utility advantages of -0.0325 and 0.0047 , both below the fixed 0.02 corrective margin. The release gate thus preserves the only corrective intervention in the fixed proposal stream and removes the harmful and negligible alternatives.

D. Reasoner-Interface Evaluation

The slow reasoner is replaced with six alternative models while the RGD gate, action schema, and fallback path remain unchanged.

Table III shows that distance and completion remain tightly aligned across all seven models: valid responses range from 112 to 138 per 30 episodes, mean distance from 599 to 603 m, and every model achieves 29 of 30 completions. Under the evaluated protocol, system-level outcomes are determined primarily by RGD’s internal gating, shared action mapping, and fallback mechanisms rather than by standalone reasoning-model performance.

E. Evaluation Across Lane and Density Conditions

Table IV reports mean distance and successful runs across six lane/density configurations without retuning the gate. At low density (2.0), successful runs reach 28–30 and mean distance 599–629 m. At high density (3.0), successful runs drop to 20–23 and mean distance decreases to 429–478 m, consistent with the increased task difficulty under dense traffic. RGD operates across the evaluated lane counts and vehicle densities with no per-setting threshold adjustment.

F. Cross-Scenario Evaluation

1) *Maneuver Classes and Traffic Density*: Following the five maneuver classes, three left-turn density levels, and 1,000 episodes per cell reported by Ma et al. [12], RGD is evaluated at 2 Hz for 30 s. Table V reports the resulting success rates. RGD achieves 0.986 on merge scenarios and maintains above 0.80 success across all three density levels. On roundabout

TABLE V
SUCCESS-RATE CONTEXT ACROSS MANEUVER AND TRAFFIC-DENSITY SETTINGS (SUCCESS RATE IN $[0, 1]$).

Method	Traffic Scenario					Left Turn Traffic Density		
	Left	Right	Straight	Merge	Round.	1.0	2.0	3.0
VLM attention allocation [12]	0.966	0.991	0.984	0.987	0.974	0.975	0.961	0.948
RGD (ours)	0.842	0.908	0.868	0.986	0.747	0.906	0.863	0.801

TABLE VI
COMPOUND-HAZARD CONTEXT IN METADRIIVE.

Method	Scene	Safety (%)	Speed (m/s)
ASR-RL [18]	A	99.2	7.1
	B	98.3	8.6
	C	97.9	6.8
RGD (ours)	A	100.0	4.8
	B	94.8	7.0
	C	84.6	6.9

and left-turn scenarios, success rates are lower than those of the VLM attention-allocation baseline, which applies scenario-specific uncertainty-weighted attention trained on those scene types. RGD uses a single scenario-agnostic release gate; the gap on complex maneuvers is consistent with the absence of scenario-specific fine-tuning.

2) *Compound Hazard Scenes*: Following the scenario and metric definitions of Wang et al. [18], Table VI reports collision-free safety rates and mean speed in three MetaDrive compound-hazard scenes. RGD attains 100% collision-free safety in Scene A (pedestrian crossing) and 94.8% in Scene B (uncontrolled intersection), with a single gate configuration and no scenario-specific policy training. In Scene C (combined vehicle–pedestrian traffic), RGD records 84.6% safety. The lower rate relative to ASR-RL is consistent with the advantage of trained scenario-specific policies in high-complexity long-tail environments; RGD uses no scenario-specific policy training or gate retuning in this evaluation.

G. Discussion

RGD converts slow reasoning from a persistent command source into a release-conditioned proposal, decoupling the decision to invoke deliberation from the decision to act on its output. The admission gate restricts requests to states where a feasible corrective alternative exists within the delay window; the release gate restricts actuation to proposals that offer a demonstrable advantage over the concurrent fast action in the evolved state.

The paired common-protocol study establishes the principal result: RGD reduces slow-path requests by over 90% relative to continual invocation while preserving per-seed task outcomes, occupying a selective operating point between the overhead of always-slow reasoning and the inflexibility of always-fast control. Component ablations show that H and N jointly govern release exposure while L and A determine which states reach the slow path, with each admission criterion playing a distinct and non-redundant role. The matched replay confirms that release validation retains the corrective proposal and suppresses the harmful and negligible alternatives.

Reasoner substitution shows that system-level performance remains stable across the evaluated reasoners, indicating that system-level outcomes are determined primarily by RGD’s internal gating, shared action mapping, and fallback mechanisms rather than by standalone model performance. Across the full range of lane counts and vehicle densities, RGD operates without threshold retuning, with the resulting mean distance and successful-run counts reported in Table IV. The MetaDrive results further support consistent gate operation across compound-hazard scenes outside the primary highway setting.

Limitations and future work. The current evaluation uses discrete high-level commands and controlled, measurable response delays. In long-tail scenarios such as dense roundabouts or mixed vehicle–pedestrian traffic, the absence of scenario-specific fine-tuning limits performance relative to methods with targeted policy libraries. RGD also depends on the slow reasoner producing structured output parseable by the shared mapper; models not instruction-tuned to the action schema may yield lower response validity. Future work will target fine-tuned driving models for rare maneuver coverage, continuous trajectory proposals with delay-aware replanning, and real-world validation beyond simulator dynamics.

V. CONCLUSION

This paper presents Recoverability-Gated Deliberation, a two-stage framework that governs when to invoke slow reasoning and whether a delayed LLM proposal may enter the closed-loop control path. By formalizing query admission and release validation as separate authority decisions, RGD ensures that a delayed proposal reaches actuation only when it is feasible, distinct from the current fast action, and offers a recovery-cost advantage in the evolved traffic state. The companion R-VoD diagnostic provides an offline measure of the corrective opportunity retained at release.

Comprehensive closed-loop experiments demonstrate that RGD substantially reduces slow-path invocations while preserving task outcomes, and that release validation retains the corrective intervention in the evaluated proposal stream while filtering harmful or negligible alternatives. The common action interface was consistently evaluated across the considered slow reasoners, traffic settings, and additional scenario families. By reconceiving slow reasoning as an expiring proposal rather than a persistent command source, RGD establishes a principled, release-aware decision interface between deliberative reasoning and real-time control.

REFERENCES

- [1] L. Wen et al., “DiLu: A knowledge-driven approach to autonomous driving with large language models,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.

- [2] Z. Xu *et al.*, "DriveGPT4: Interpretable end-to-end autonomous driving via large language model," *IEEE Robot. Autom. Lett.*, vol. 9, no. 10, pp. 8186–8193, 2024.
- [3] H. Shao *et al.*, "LMDrive: Closed-loop end-to-end driving with large language models," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2024, pp. 15 120–15 130.
- [4] C. Sima *et al.*, "DriveLM: Driving with graph visual question answering," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2024, pp. 256–274.
- [5] J. Mao, Y. Qian, J. Ye, H. Zhao, and Y. Wang, "GPT-Driver: Learning to drive with GPT," arXiv:2310.01415, 2023.
- [6] Y. Hu, D. Ou, J. Huang, M. Wu, M. Hao, and R. Yu, "Integrating vision and language foundation models for enhanced navigation and decision-making in connected autonomous vehicles," *IEEE Trans. Veh. Technol.*, vol. 74, no. 10, pp. 16 233–16 249, 2025.
- [7] D. Pei, J. He, K. Liu, Y. Wu, Y. Lei, and X. Xiao, "Enhancement of large language models driving knowledge for practical autonomous driving decision making," *IEEE Trans. Veh. Technol.*, 2026, early access, doi: 10.1109/TVT.2026.3678726.
- [8] R. Xin *et al.*, "NetRoller: Interfacing general and specialized models for end-to-end autonomous driving," *IEEE Trans. Veh. Technol.*, vol. 75, no. 5, pp. 7469–7482, 2026.
- [9] K. Qian *et al.*, "FASIONAD: Fast and slow fusion thinking systems for human-like autonomous driving with adaptive feedback," arXiv:2411.18013, 2024.
- [10] R. Zhang *et al.*, "AdaDrive: Self-adaptive slow-fast system for language-grounded autonomous driving," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2025, pp. 5112–5121.
- [11] X. Chen, X. Wang, W. Chen, and J. Gao, "An adaptive multimodal end-to-end autonomous driving framework based on behavior discrepancy," *IEEE Trans. Intell. Transp. Syst.*, 2026, early access, doi: 10.1109/TITS.2026.3706626.
- [12] B. Ma, H. Liu, R. Zhong, P. Liu, X. Zhou, and J. Ma, "Behavioral uncertainty-aware attention allocation via VLMs for interactive autonomous driving," *IEEE Trans. Veh. Technol.*, 2026, early access, doi: 10.1109/TVT.2026.3713420.
- [13] Y. Chen, Z.-H. Ding, Z. Wang, Y. Wang, L. Zhang, and S. Liu, "Asynchronous large language model enhanced planner for autonomous driving," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2024, pp. 22–38.
- [14] L. Li, J. Fang, J. Xue, and C. Lv, "Vision-language model-enabled dual-system for autonomous driving in safety-critical transportation," *IEEE Trans. Intell. Transp. Syst.*, 2026, early access, doi: 10.1109/TITS.2026.3708214.
- [15] S. Chen, Y. Sun, D. Li, Q. Wang, Q. Hao, and J. Sifakis, "Runtime safety assurance for learning-enabled control of autonomous driving vehicles," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2022, pp. 8978–8984.
- [16] L. Chen *et al.*, "Driving with LLMs: Fusing object-level vector modality for explainable autonomous driving," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2024, pp. 14 093–14 100.
- [17] R. Zhao *et al.*, "VLM-Driver: Human-like autonomous driving decision-making via vision language model," *IEEE Trans. Veh. Technol.*, vol. 75, no. 5, pp. 7327–7342, 2026.
- [18] J. Wang, H. Ren, X. Zhu, and Z. Ma, "Enhancing autonomous vehicle decision-making through policy transfer with large language model," *IEEE Trans. Intell. Transp. Syst.*, 2025, early access, doi: 10.1109/TITS.2025.3565762.
- [19] Z. Cui *et al.*, "C-TRAIL: A commonsense world framework for trajectory planning in autonomous driving," *IEEE Trans. Veh. Technol.*, 2026, early access, doi: 10.1109/TVT.2026.3682668.
- [20] X. Wang, Z. Zhu, G. Huang, X. Chen, J. Zhu, and J. Lu, "DriveDreamer: Towards real-world-drive world models for autonomous driving," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2024, pp. 55–72.
- [21] C. Min *et al.*, "DriveWorld: 4D pre-trained scene understanding via world models for autonomous driving," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2024, pp. 15 522–15 533.
- [22] W. Zheng, R. Song, X. Guo, C. Zhang, and L. Chen, "GenAD: Generative end-to-end autonomous driving," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2024, pp. 87–104.
- [23] B. Liao *et al.*, "DiffusionDrive: Truncated diffusion model for end-to-end autonomous driving," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2025, pp. 12 037–12 047.
- [24] J. Mei *et al.*, "Continuously learning, adapting, and improving: A dual-process approach to autonomous driving," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 37, 2024, pp. 123 261–123 290.
- [25] Y. Ma *et al.*, "LeapVAD: A leap in autonomous driving via cognitive perception and dual-process thinking," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 37, no. 4, pp. 1963–1977, 2026.
- [26] Y. Luo *et al.*, "AdaThinkDrive: Adaptive thinking via reinforcement learning for autonomous driving," arXiv:2509.13769, 2025.
- [27] S. Russell and E. Wefald, "Principles of metareasoning," *Artif. Intell.*, vol. 49, no. 1–3, pp. 361–395, 1991.
- [28] M. Boddy and T. L. Dean, "Deliberation scheduling for problem solving in time-constrained environments," *Artif. Intell.*, vol. 67, no. 2, pp. 245–285, 1994.
- [29] S. Zilberstein, "Using anytime algorithms in intelligent systems," *AI Mag.*, vol. 17, no. 3, pp. 73–83, 1996.
- [30] B. Cserna, W. Ruml, and J. Frank, "Planning time to think: Metareasoning for on-line planning with durative actions," in *Proc. Int. Conf. Autom. Plan. Scheduling (ICAPS)*, vol. 27, 2017, pp. 56–60.
- [31] A. Elboher, A. Bensoussan, E. Karpas, W. Ruml, S. S. Shperberg, and E. Shimony, "A formal metareasoning model of concurrent planning and execution," in *Proc. AAAI Conf. Artif. Intell.*, vol. 37, no. 10, 2023, pp. 12 427–12 435.
- [32] P. Tabuada, "Event-triggered real-time scheduling of stabilizing control tasks," *IEEE Trans. Autom. Control*, vol. 52, no. 9, pp. 1680–1685, 2007.
- [33] M. Li, J. Gao, L. Zhao, and X. Shen, "Adaptive computing scheduling for edge-assisted autonomous driving," *IEEE Trans. Veh. Technol.*, vol. 70, no. 6, pp. 5318–5331, 2021.
- [34] T. G. Molnar, A. K. Kiss, A. D. Ames, and G. Orosz, "Safety-critical control with input delay in dynamic environment," *IEEE Trans. Control Syst. Technol.*, vol. 31, no. 4, pp. 1507–1520, 2023.
- [35] S. Shalev-Shwartz, S. Shammah, and A. Shashua, "On a formal model of safe and scalable self-driving cars," arXiv:1708.06374, 2017.
- [36] A. Liniger and J. Lygeros, "Real-time control for autonomous racing based on viability theory," *IEEE Trans. Control Syst. Technol.*, vol. 27, no. 2, pp. 464–478, 2019.
- [37] E. Leurent, "An environment for autonomous driving decision-making," GitHub repository, 2018, version 1.12. [Online]. Available: <https://github.com/Farama-Foundation/HighwayEnv>
- [38] Q. Li, Z. Peng, L. Feng, Q. Zhang, Z. Xue, and B. Zhou, "MetaDrive: Composing diverse driving scenarios for generalizable reinforcement learning," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 3, pp. 3461–3475, 2023.
- [39] A. Yang *et al.*, "Qwen3 technical report," arXiv:2505.09388, 2025.
- [40] G. Kou *et al.*, "PADriver: Towards personalized autonomous driving," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, 2025, pp. 1–8.